

Exception Handling, Comprehensions and Decorators in Python

Contents

1	Introduction	4
2	What is Exception?	4
3	Simple Example	4
4	Basic try-except	5
5	How try Works	5
6	Exception Object	5
7	Specific Exception Handling	5
8	Why Specific Exceptions are Better	6
9	Infinite Input Validation Loop	6
10	What is else in Exception Handling?	6
11	Program Flow	6
12	finally Block	7
13	Example	7
14	File Exception	7
15	Real-world Importance	7
16	Comprehensions	8
17	List Comprehension	8
18	Structure of List Comprehension	8
19	Filtering with Comprehension	8

20 Three Important Parts	8
21 Odd Numbers	9
22 Dictionary Comprehension	9
23 What Happened Here?	9
24 Conditional Expression in Comprehension	9
25 Decorator Introduction	9
26 Basic Decorator Example	10
27 What Happens Here?	10
28 @ Symbol	10
29 Decorator Flow	10
30 Execution Time Decorator	10
31 Using time Module	11
32 Calculating Duration	11
33 *args and **kwargs in Decorators	11
34 Example	11
35 Real-world Use of Decorators	11
36 Important Final Thought	12
37 Exercises	13
37.1 Exercise 1	13
37.2 Exercise 2	13
37.3 Exercise 3	13
37.4 Exercise 4	13
37.5 Exercise 5	13
37.6 Exercise 6	13
37.7 Exercise 7	13
37.8 Exercise 8	13
37.9 Exercise 9	14
37.10Exercise 10	14
38 Challenge Questions	15
38.1 Challenge 1	15
38.2 Challenge 2	15
38.3 Challenge 3	15
38.4 Challenge 4	15

38.5	Challenge 5		15
38.6	Challenge 6		15
38.7	Challenge 7		15
38.8	Challenge 8		15
38.9	Challenge 9		15
38.10	Challenge 10		15

1 Introduction

Till now, our programs worked nicely when:

- users entered correct input
- files existed properly
- values were valid

But real users are unpredictable.
Someone will eventually type:

```
hello
```

where your program expected:

```
123
```

And then:

Boom. Program crashes.

This is where:

Exception Handling

becomes important.

2 What is Exception?

Exception means:

Runtime error

Examples:

- dividing by zero
- wrong input type
- missing file
- invalid operation

3 Simple Example

```
1 a = 5
```

This is:

Syntax Error

Program cannot even start.

But this:

```
int("hello")
```

is:

Runtime Error (Exception)

Program starts, then crashes later.

4 Basic try-except

```
try:
    user_input = int(input("Enter number: "))
except:
    print("Something went wrong")
```

5 How try Works

Python says:

“Try running this code.”

If error occurs:

- stop normal flow
- jump to except block

6 Exception Object

```
except Exception as e:
```

```
e
```

stores actual error message.

Example:

```
invalid literal for int()
```

7 Specific Exception Handling

```
except ValueError:
```

Handles only:

- invalid value conversion errors

Example:

```
int("hello")
```

8 Why Specific Exceptions are Better

Bad practice:

```
except:
```

Good practice:

```
except ValueError:
```

Because:

- debugging becomes easier
- we know exact problem

9 Infinite Input Validation Loop

```
while True:

    try:
        user_input = int(input())

    except ValueError:
        print("Enter number only")

    else:
        break
```

10 What is else in Exception Handling?

```
else:
```

runs only if:

- no exception occurred

11 Program Flow

```
try -> exception?
```

If YES:

- go to except

If NO:

- go to else

12 finally Block

```
finally:
```

always runs.

No matter:

- error happened
- no error happened

13 Example

```
finally:  
    print("Program completed")
```

Useful for:

- cleanup
- closing resources

14 File Exception

```
except FileNotFoundError:
```

Handles:

- missing file

Example:

```
open("missing.txt")
```

15 Real-world Importance

Exception handling is used everywhere:

- banking apps
- APIs
- games
- automation
- backend systems

Without exception handling:

Software becomes fragile.

16 Comprehensions

Comprehension means:

Short and smart way to create collections

17 List Comprehension

Normal way:

```
int_marks = []

for mark in marks:
    int_marks.append(int(mark))
```

Short version:

```
int_marks = [
    int(mark)
    for mark in marks
]
```

18 Structure of List Comprehension

```
[new_value for item in collection]
```

19 Filtering with Comprehension

```
even_numbers = [
    num
    for num in nums
    if num % 2 == 0
]
```

20 Three Important Parts

1. What to keep?
2. Where from?
3. On what condition?

21 Odd Numbers

```
odd_numbers = [  
    num  
    for num in nums  
    if num % 2 != 0  
]
```

22 Dictionary Comprehension

```
emotion_to_life = {  
    value:key  
    for key, value in life_to_emotion.items()  
}
```

23 What Happened Here?

Keys and values got reversed.

Original:

```
5 -> Happy
```

New dictionary:

```
Happy -> 5
```

24 Conditional Expression in Comprehension

```
result = [  
    "Even"  
    if num % 2 == 0  
    else "Odd"  
  
    for num in nums  
]
```

25 Decorator Introduction

Decorator means:

Function that modifies another function

Sounds magical.

Actually:

It's just functions using functions.

26 Basic Decorator Example

```
def deco(func):  
    def wrapper_func():  
        print("Before")  
        func()  
        print("After")  
    return wrapper_func
```

27 What Happens Here?

Decorator:

- adds extra behavior
- before and after function execution

28 @ Symbol

```
@deco
```

is shortcut for:

```
just_func = deco(just_func)
```

29 Decorator Flow

```
@deco  
def just_func():
```

means:

1. send function into decorator
2. decorator wraps function
3. returns enhanced function

30 Execution Time Decorator

```
def show_execution_time(func):
```

This decorator measures:

- how long function takes

31 Using time Module

```
start = time.time()
```

stores starting time.

```
end = time.time()
```

stores ending time.

32 Calculating Duration

```
end - start
```

gives total execution time.

33 *args and **kwargs in Decorators

```
def wrapper_func(*args, **kwargs):
```

Why needed?

Because decorated functions may have:

- different parameters

34 Example

```
heavy_task_func(a=0, b=1)
```

Decorator should support:

- positional arguments
- keyword arguments

35 Real-world Use of Decorators

Decorators are heavily used in:

- Django
- Flask
- FastAPI
- Authentication systems
- Logging
- Performance monitoring

36 Important Final Thought

Earlier, we wrote:

- simple scripts

Now we are learning:

- robust programming
- reusable logic
- smart data processing

This is the transition from:

“learning syntax”

to:

“thinking like programmer”

37 Exercises

37.1 Exercise 1

Take number input safely using:

- try
- except

37.2 Exercise 2

Handle division by zero error.

37.3 Exercise 3

Take filename input and safely read file.

37.4 Exercise 4

Convert list of string marks into integer list using comprehension.

37.5 Exercise 5

Create list of squares from:

```
1 to 20
```

using comprehension.

37.6 Exercise 6

Create list of even numbers only.

37.7 Exercise 7

Create dictionary comprehension:

```
number -> square
```

Example:

```
1 -> 1
2 -> 4
3 -> 9
```

37.8 Exercise 8

Create decorator printing:

```
Function started
Function ended
```

37.9 Exercise 9

Create decorator counting function calls.

37.10 Exercise 10

Create decorator measuring execution time of:

```
sum()
```

on large list.

38 Challenge Questions

38.1 Challenge 1

Create retry system:

- max 3 invalid inputs

38.2 Challenge 2

Create login system using exception handling.

38.3 Challenge 3

Create nested comprehension for matrix.

38.4 Challenge 4

Create decorator requiring password before function execution.

38.5 Challenge 5

Create decorator logging all function calls into file.

38.6 Challenge 6

Create decorator caching function results.

38.7 Challenge 7

Create custom exception:

```
AgeTooLowError
```

38.8 Challenge 8

Create comprehension generating:

- prime numbers

38.9 Challenge 9

Create decorator working with:

- any function
- any parameter count

38.10 Challenge 10

Create mini framework-like routing decorator.